

Installation & Instructions

This guide provides end-to-end instructions for installing, configuring, running, and deploying the Booking Platform REST API.

System Prerequisites

Ensure you have the following installed on your machine:

- **Node.js:** v24.18.0 or higher (Active LTS)
 - **npm:** v10.x or higher
 - **Docker & Docker Compose:** (For containerized runs and production deployments)
 - **Git:** (For version control)
-

Step 1: Clone the Repository

Clone the project repository to your local machine:

```
git clone https://github.com/MircoFernando/booking-platform-api.git
cd booking-platform-api
```

Step 2: Environment Configuration

Create a `.env` file in the project root directory by copying the provided `.env.example`:

```
cp .env.example .env
```

Open the newly created `.env` file and set the configuration parameters:

```
# Application Host Port
PORT=3000
NODE_ENV=development

# Database Settings (PostgreSQL)
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USER=postgres
DATABASE_PASSWORD=your_secure_password
DATABASE_NAME=booking_platform

# Prisma Database connection URL
DATABASE_URL="postgresql://${DATABASE_USER}:${DATABASE_PASSWORD}@${DATABASE_HOST}:${DATABASE_PORT}/${DATABASE_NAME}?schema=public"

# JSON Web Token Secret Keys
JWT_SECRET=super_secret_access_key_change_me
JWT_REFRESH_SECRET=super_secret_refresh_key_change_me

# Docker Registry Settings
DOCKER_REGISTRY_USER=your_dockerhub_username // Only if pushing to DockerHub
```

Step 3: Running the Application

Option A: Local Development using Docker Compose (Recommended)

This approach sets up a local PostgreSQL database container and builds your Node NestJS server in watch mode inside Docker.

1. Launch the Docker stack:

```
docker compose up -d --build
```

2. Check container status:

```
docker compose ps
```

3. Verify the database state:

The compose stack automatically triggers pending migrations at boot. You can explore the database visually by running Prisma Studio:

```
npx prisma studio
```

Open <http://localhost:5555> to view user data.

Option B: Local Running without Docker (Bare Metal)

Follow these steps if you want to run Node.js directly on your local system:

1. Install Node Packages:

```
npm install --legacy-peer-deps
```

2. Start a local Postgres database:

You can spin up just the database container from the compose settings:

```
docker compose up -d db
```

3. Sync Database Schema:

Generate the Prisma Client client mapping and apply migrations directly:

```
npx prisma generate
npx prisma migrate dev --name init
```

4. Launch the Server:

```
# Development (Hot reload active)
npm run start:dev

# Production build & start
npm run build
npm run start:prod
```



Production Deployment & Live Environment

I have deployed the Booking Platform REST API to a production environment hosted on **DigitalOcean**.

I configured a continuous deployment (CD) pipeline using **GitHub Actions** that triggers automatically on every push to the `main` branch. The pipeline:

1. Compiles the codebase and builds the production-optimized multi-stage Docker container.
2. Pushes the Docker image to Docker Hub.
3. Securely SSHs into the DigitalOcean Droplet, pulls the latest Docker image, applies database schema migrations (`npm run prisma migrate deploy`), and restarts the running containers.
4. Cleans up old, unused images on the host to maintain a stable storage footprint.

Live Production URL

You can access and test the live application directly: <http://178.128.17.103:3000/docs>